МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнил: А. А. Вольвач
Группа: М8О-204БВ-25
Преподаватель: Н. А. Гапонов
Дата: 22.10.2026
Оценка: _____

Москва, 2026

Условие

Цель работы: приобретение практических навыков в управлении процессами в операционной системе и в обеспечении обмена данными между процессами посредством каналов (pipe).

Задание: составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должна создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы. Работа выполнена для операционной системы Windows с использованием Win32 API.

Группа вариантов 2: родительский процесс создаёт дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в pipe1. Родительский процесс читает из pipe1 и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

Вариант 7: в файле записаны команды вида «число число число<endline>». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип float. Количество чисел может быть произвольным.

Метод решения

Решение состоит из двух отдельных программ: parent.exe (родительский процесс) и child.exe (дочерний процесс). Схема их взаимодействия показана на рисунке 1.

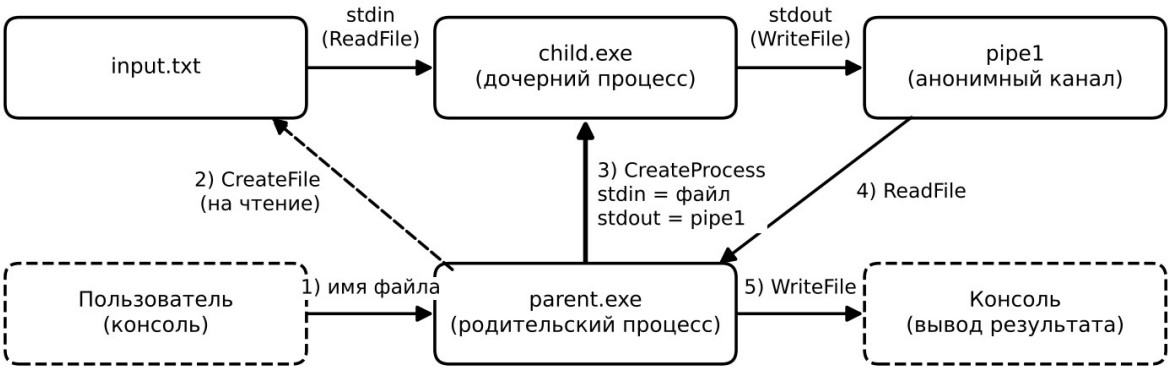


Рисунок 1 – Схема взаимодействия процессов

Дочерний процесс ничего не знает ни о файле, ни о канале: это обычный фильтр, который читает данные из стандартного ввода и пишет результат в стандартный вывод. Подмену стандартных потоков выполняет родитель при вызове CreateProcessA: в структуре STARTUPINFOA он задаёт дескрипторы в полях hStdInput (открытый файл) и hStdOutput (записывающий конец pipe1) и флаг STARTF_USESTDHANDLES, а параметр bInheritHandles = TRUE разрешает дочернему процессу унаследовать эти дескрипторы. Это аналог связки fork + dup2 + exec в Linux. Благодаря такой архитектуре child.exe можно запустить и проверить отдельно, например командой Get-Content input.txt | .\child.exe.

Алгоритм родительского процесса:

1) прочитать с консоли первую строку – имя файла;

2) открыть файл на чтение (CreateFileA) с наследуемым дескриптором;

3) создать анонимный канал pipe1 (CreatePipe); у читающего конца снять признак наследования (SetHandleInformation), так как он нужен только родителю;

4) заполнить STARTUPINFOA (stdin = файл, stdout = pipe1, stderr = консоль родителя) и запустить child.exe (CreateProcessA);

5) закрыть у себя дескриптор файла и записывающий конец pipe1 – теперь они есть только у дочернего процесса;

6) в цикле читать данные из pipe1 (ReadFile) и выводить их в свой стандартный вывод (WriteFile), пока ReadFile не вернёт ERROR_BROKEN_PIPE – признак того, что все записывающие концы канала закрыты;

7) дождаться завершения дочернего процесса (WaitForSingleObject), получить его код возврата (GetExitCodeProcess) и закрыть все дескрипторы.

Алгоритм дочернего процесса: стандартный ввод читается системным вызовом ReadFile блоками по 4096 байт и разбирается посимвольно. Символы, отличные от пробела, табуляции и перевода строки, накапливаются в буфере текущего числа (до 64 символов). На разделителе число преобразуется функцией strtof и прибавляется к сумме строки (тип float); на символе перевода строки результат записывается в стандартный вывод (то есть в pipe1). Строка целиком в памяти не хранится, поэтому количество чисел в строке не ограничено. Пустые строки пропускаются, некорректное число или переполнение float приводят к сообщению об ошибке для этой строки. Последняя строка файла обрабатывается и без завершающего перевода строки. Работа заканчивается, когда ReadFile возвращает 0 байт (конец файла).

Источники: документация Microsoft Learn – «Creating a Child Process with Redirected Input and Output» (learn.microsoft.com/en-us/windows/win32/procthread/creating-a-child-process-with-redirected-input-and-output); справочник по функциям CreatePipe, CreateProcessA, SetHandleInformation, ReadFile, WriteFile (learn.microsoft.com/en-us/windows/win32/api); документация Process Monitor (learn.microsoft.com/en-us/sysinternals/downloads/procmon).

Описание программы

Проект состоит из двух программ на языке Си и общего модуля с вспомогательными функциями; сборка выполняется утилитой mingw32-make (компилятор GCC из пакета MinGW-w64) по Makefile. Оба исполняемых файла создаются в одном каталоге: CreateProcessA ищет child.exe в первую очередь в каталоге, из которого загружен parent.exe. Полный исходный код приведён в приложении А.

Структура файлов

Таблица 1 – Файлы проекта

Файл	Назначение
parent.c	родительский процесс: читает имя файла, открывает файл, создаёт pipe1, запускает child.exe с перенаправленными stdin/stdout, выводит данные из pipe1, ждёт завершения дочернего процесса
child.c	дочерний процесс: читает команды из stdin, считает сумму чисел каждой строки (float) и пишет результат в stdout
common.h, common.c	общие функции: вывод текста системной ошибки (print_error) и запись буфера целиком (write_all)
Makefile	сборка parent.exe и child.exe (цели all и clean)
input.txt	тестовый файл с командами

Основные типы данных и функции

Используемые типы Win32 API:

– HANDLE – дескриптор объекта ядра (файл, конец канала, процесс, поток);

– SECURITY_ATTRIBUTES – атрибуты создаваемого объекта; поле bInheritHandle = TRUE делает дескриптор наследуемым;

– STARTUPINFOA – параметры запуска процесса; поля hStdInput, hStdOutput, hStdError задают стандартные потоки потомка, флаг STARTF_USESTDHANDLES включает их использование;

– PROCESS_INFORMATION – результат CreateProcessA: дескрипторы процесса и его главного потока (hProcess, hThread) и их идентификаторы (dwProcessId, dwThreadId).

Собственные типы дочернего процесса:

– Reader – буфер чтения стандартного ввода: дескриптор h, массив buf[4096], позиция pos и число прочитанных байт len;

– LineState – состояние текущей строки: сумма sum (float), количество чисел count, признак ошибки error, текст текущего числа token[65] и его длина tlen.

Функции:

– void print_error(const char *context) – получает код ошибки GetLastError(), текст ошибки FormatMessageW, перекодирует его в кодировку консоли и печатает в stderr;

– BOOL write_all(HANDLE h, const char *data, DWORD len) – вызывает WriteFile в цикле, пока не будут записаны все len байт; возвращает FALSE при ошибке;

– static int read_file_name(char *buf, size_t size) (parent.c) – читает имя файла из консоли, отрезает перевод строки, проверяет, что имя не пустое;

– static int reader_getc(Reader *r, char *c) (child.c) – возвращает очередной символ stdin: 1 – символ прочитан, 0 – конец файла, –1 – ошибка ReadFile;

– static void finish_token(LineState *st, unsigned long line_no) (child.c) – преобразует накопленное число strtof, проверяет корректность и прибавляет к сумме;

– static BOOL finish_line(LineState *st, HANDLE out, unsigned long line_no) (child.c) – формирует строку результата и записывает её в stdout;

– static void log_stderr(const char *fmt, ...) (child.c) – выводит сообщение в stderr одним вызовом WriteFile, чтобы строка не перемешивалась с выводом родителя.

Обработка ошибок: результат каждого вызова Win32 API проверяется; при ошибке выводится сообщение с текстом и кодом GetLastError(), выполняется переход на метку cleanup, где закрываются все полученные дескрипторы и, если дочерний процесс уже создан, родитель дожидается его завершения. Код возврата дочернего процесса передаётся в код возврата родителя.

Используемые системные вызовы

Функции Win32 API являются обёртками над системными вызовами ядра Windows (Native API); соответствующий вызов ядра указан в скобках.

Таблица 2 – Используемые системные вызовы

Системный вызов	Назначение в программе
CreateFileA (NtCreateFile)	открытие входного файла на чтение (GENERIC_READ, OPEN_EXISTING) с наследуемым дескриптором
CreatePipe (NtCreateNamedPipeFile, NtOpenFile)	создание анонимного канала pipe1: дескриптор для чтения и дескриптор для записи
SetHandleInformation (NtSetInformationObject)	снятие флага HANDLE_FLAG_INHERIT с читающего конца pipe1

Системный вызов	Назначение в программе
CreateProcessA (NtCreateUserProcess)	создание дочернего процесса child.exe с наследованием дескрипторов и подменой stdin/stdout
GetStdHandle	получение дескрипторов стандартных потоков (берутся из параметров процесса, без перехода в ядро): в child – файл и pipe1, в parent – консоль
ReadFile (NtReadFile)	parent – чтение из pipe1; child – чтение команд из stdin (файла)
WriteFile (NtWriteFile)	parent – вывод в консоль; child – запись результата в stdout (pipe1) и сообщений в stderr
WaitForSingleObject (NtWaitForSingleObject)	ожидание завершения дочернего процесса
GetExitCodeProcess (NtQueryInformationProcess)	получение кода возврата дочернего процесса
CloseHandle (NtClose)	закрытие дескрипторов файла, концов канала, процесса и потока
ExitProcess (NtTerminateProcess)	завершение процесса (вызывается библиотекой Си при возврате из main)
GetLastError, FormatMessageW, GetCurrentProcessId	диагностика: код и текст ошибки, PID процесса для вывода

Ключевые фрагменты кода

Родитель открывает файл и создаёт канал. Структура SECURITY_ATTRIBUTES с bInheritHandle = TRUE делает дескрипторы наследуемыми; у читающего конца канала наследование затем отключается.

Листинг 1 – Открытие файла и создание канала (parent.c)

```
    /* дескрипторы с этими атрибутами наследуются дочерним процессом */
    sa.nLength = sizeof(sa);
    sa.lpSecurityDescriptor = NULL;
    sa.bInheritHandle = TRUE;

    hFile = CreateFileA(file_name, GENERIC_READ, FILE_SHARE_READ, &sa,
                        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        print_error("CreateFile");
        goto cleanup;
    }

    if (!CreatePipe(&hPipeRead, &hPipeWrite, &sa, 0)) {
        print_error("CreatePipe");
        goto cleanup;
    }
    /* конец для чтения остаётся только у родителя */
    if (!SetHandleInformation(hPipeRead, HANDLE_FLAG_INHERIT, 0)) {
        print_error("SetHandleInformation");
        goto cleanup;
    }
```

Подмена стандартных потоков и запуск дочернего процесса. Флаг STARTF_USESTDHANDLES указывает, что потомок должен использовать дескрипторы из полей hStd*.

Листинг 2 – Запуск дочернего процесса с перенаправлением потоков (parent.c)

```
    /* stdin ребёнка - файл, stdout - pipe1 */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = hFile;
    si.hStdOutput = hPipeWrite;
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);

    if (!CreateProcessA(NULL, cmd_line, NULL, NULL, TRUE, 0, NULL, NULL, &si,
&pi)) {
        print_error("CreateProcess(" CHILD_EXE ")");
        goto cleanup;
    }
```

Сразу после запуска родитель закрывает свои копии дескрипторов, переданных потомку. Если оставить у себя записывающий конец pipe1, канал никогда не будет считаться закрытым и ReadFile в цикле ниже заблокируется навсегда (взаимоблокировка).

Листинг 3 – Закрытие лишних дескрипторов и чтение из канала (parent.c)

```
    /* закрываем свои копии, иначе ReadFile не дождётся конца канала */
    CloseHandle(hPipeWrite);
    hPipeWrite = NULL;
    CloseHandle(hFile);
    hFile = INVALID_HANDLE_VALUE;

    hStdout = GetStdHandle(STD_OUTPUT_HANDLE);
    if (hStdout == INVALID_HANDLE_VALUE) {
        print_error("GetStdHandle(STD_OUTPUT_HANDLE)");
        goto cleanup;
    }
    for (;;) {
        DWORD n = 0;
        if (!ReadFile(hPipeRead, buf, sizeof(buf), &n, NULL)) {
            if (GetLastError() == ERROR_BROKEN_PIPE) {
                break;  /* ребёнок закрыл канал */
            }
            print_error("ReadFile(pipe1)");
            goto cleanup;
        }
        if (n == 0) {
            break;
        }
        if (!write_all(hStdout, buf, n)) {
            print_error("WriteFile(stdout)");
            goto cleanup;
        }
    }
```

Ожидание дочернего процесса выполняется только после того, как канал прочитан до конца: иначе при большом объёме вывода потомок заблокировался бы на заполненном буфере канала, а родитель – в ожидании его завершения. В блоке cleanup освобождаются все ресурсы, в том числе при выходе по ошибке.

Листинг 4 – Ожидание дочернего процесса и освобождение ресурсов (parent.c)

```
    if (WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_FAILED) {
        print_error("WaitForSingleObject");
        goto cleanup;
    }
    if (!GetExitCodeProcess(pi.hProcess, &child_code)) {
        print_error("GetExitCodeProcess");
        goto cleanup;
    }
    printf("[parent PID %lu] child PID %lu exited with code %lu\n",
           (unsigned long)GetCurrentProcessId(), (unsigned long)pi.dwProcessId,
           (unsigned long)child_code);
    result = (child_code == 0) ? EXIT_SUCCESS : EXIT_FAILURE;

cleanup:
    if (hPipeWrite != NULL)            CloseHandle(hPipeWrite);
    if (hPipeRead != NULL)             CloseHandle(hPipeRead);
    if (hFile != INVALID_HANDLE_VALUE) CloseHandle(hFile);
    if (pi.hThread != NULL)            CloseHandle(pi.hThread);
    if (pi.hProcess != NULL) {
        WaitForSingleObject(pi.hProcess, INFINITE);
        CloseHandle(pi.hProcess);
    }
    return result;
```

Дочерний процесс читает стандартный ввод системным вызовом ReadFile блоками по BUF_SIZE = 4096 байт и отдаёт данные по одному символу.

Листинг 5 – Буферизованное чтение stdin (child.c)

```
/* 1 - символ прочитан, 0 - конец файла, -1 - ошибка */
static int reader_getc(Reader *r, char *c)
{
    if (r->pos == r->len) {
        if (!ReadFile(r->h, r->buf, sizeof(r->buf), &r->len, NULL)) {
            if (GetLastError() == ERROR_BROKEN_PIPE) {
                return 0;
            }
            return -1;
        }
        r->pos = 0;
        if (r->len == 0) {
            return 0;
        }
    }
    *c = r->buf[r->pos++];
    return 1;
}
```

Главный цикл дочернего процесса: получение дескрипторов stdin/stdout, переданных родителем, и посимвольный разбор входных данных.

Листинг 6 – Основной цикл дочернего процесса (child.c)

```
    /* stdin и stdout, которые задал родитель (файл и pipe1) */
    in.h = GetStdHandle(STD_INPUT_HANDLE);
    hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    if (in.h == INVALID_HANDLE_VALUE || in.h == NULL) {
        print_error("GetStdHandle(STD_INPUT_HANDLE)");
        return EXIT_FAILURE;
    }
    if (hOut == INVALID_HANDLE_VALUE || hOut == NULL) {
        print_error("GetStdHandle(STD_OUTPUT_HANDLE)");
        return EXIT_FAILURE;
    }
    in.pos = 0;
    in.len = 0;
    line_reset(&st);

    while ((r = reader_getc(&in, &c)) == 1) {
        if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
            finish_token(&st, line_no);
            if (c == '\n') {
                if (!finish_line(&st, hOut, line_no)) {
                    return EXIT_FAILURE;
                }
                line_no++;
            }
        } else if (st.tlen < TOKEN_MAX) {
            st.token[st.tlen++] = c;
        } else {
            st.too_long = 1;
        }
    }
    if (r < 0) {
        print_error("ReadFile(stdin)");
        return EXIT_FAILURE;
    }

    /* последняя строка без '\n' */
    finish_token(&st, line_no);
    if (!finish_line(&st, hOut, line_no)) {
        return EXIT_FAILURE;
```

Результаты

Программа собирается командой mingw32-make без предупреждений компилятора (ключи -Wall -Wextra). parent.exe запускается из каталога проекта, где рядом лежит child.exe. В выводе показаны PID обоих процессов: строки с префиксом [child PID ...] сформированы дочерним процессом и получены родителем через pipe1; сообщение «invalid number» дочерний процесс пишет напрямую в stderr, поэтому оно появляется раньше результатов, прошедших через канал, – процессы выполняются параллельно.

Листинг 7 – Содержимое файла input.txt

```
1 2 3
1.5 2.25 -0.75

10
0.1 0.2 0.3 0.4
  7   8<TAB>9
3.4e38 3.4e38
1 abc 2
100 200
```

Листинг 8 – Пример сеанса работы

```
PS C:\Users\User\Desktop\lab2> .\parent.exe
[parent PID 33872] started
[parent PID 33872] Enter file name: input.txt
[parent PID 33872] created child PID 28624
[child PID 28624] started
[child PID 28624] line 8: invalid number 'abc'
[child PID 28624] line 1: sum = 6
[child PID 28624] line 2: sum = 3
[child PID 28624] line 4: sum = 10
[child PID 28624] line 5: sum = 1
[child PID 28624] line 6: sum = 24
[child PID 28624] line 7: ERROR (float overflow)
[child PID 28624] line 8: ERROR (invalid input)
[child PID 28624] line 9: sum = 300
[parent PID 33872] child PID 28624 exited with code 0
```

Листинг 9 – Обработка ошибки: файл не существует

```
PS C:\Users\User\Desktop\lab2> .\parent.exe
[parent PID 12576] started
[parent PID 12576] Enter file name: nofile.txt
[PID 12576] ERROR: CreateFile: Не удается найти указанный файл. (code 2)
```

Результаты проверки на различных входных данных приведены в таблице 3.

Таблица 3 – Тестовые сценарии

№	Входные данные	Вывод	Что проверяется
1	«1 2 3»	sum = 6	базовый случай
2	«1.5 2.25 -0.75»	sum = 3	дробные и отрицательные числа
3	пустая строка	вывода нет	пустые строки пропускаются
4	«10»	sum = 10	строка из одного числа
5	«0.1 0.2 0.3 0.4»	sum = 1	сложение в типе float
6	« 7 8<TAB>9 »	sum = 24	несколько пробелов, табуляция, пробелы по краям
7	«3.4e38 3.4e38»	ERROR (float overflow)	переполнение типа float

№	Входные данные	Вывод	Что проверяется
8	«1 abc 2»	invalid number 'abc' (stderr), ERROR (invalid input)	некорректное число
9	«100 200» без перевода строки в конце файла	sum = 300	последняя строка без \n
10	имя файла nofile.txt	ERROR: CreateFile … (code 2), код возврата 1	обработка ошибки системного вызова

Особенности реализации и ограничения:

– дробная часть чисел отделяется точкой (используется локаль «C»), запись «1,5» считается некорректной;

– длина одной записи числа ограничена 64 символами, длина строки и количество чисел – не ограничены;

– имя файла вводится в кодировке консоли, поэтому файлы следует называть латиницей;

– если родитель завершится раньше дочернего процесса, запись в pipe1 в дочернем процессе вернёт ошибку, и он завершится с кодом 1 (обработка обрыва канала).

Демонстрация работы Process Monitor

Работа выполнена в Windows, где утилита strace недоступна, поэтому вместо неё использован Process Monitor из набора Sysinternals. Он регистрирует события файловой системы, реестра, процессов и потоков. В отличие от strace, он показывает не все системные вызовы, а операции, дошедшие до файловой системы и диспетчера процессов: например, вызов CreateProcessA отображается как событие «Process Create». Запись велась с фильтром «Process Name is parent.exe» и «Process Name is child.exe»; программа запущена из каталога C:\Users\User\Desktop\lab2\, введено имя input.txt. В журнале 263 события, из них 194 – обращения к реестру при загрузке библиотек. Основные события приведены в таблице 4, журнал без обращений к реестру – в приложении Б.

Таблица 4 – Хронология основных событий

Время	Процесс (PID)	Событие	Комментарий
13:49:15.572	parent.exe (24216)	Process Start	запуск родителя из терминала (Parent PID 26124)
13:49:15.58 – 13:49:26.80	parent.exe (24216)	нет событий	ожидание ввода имени файла (fgets), ≈11 с
13:49:26.803	parent.exe (24216)	CreateFile input.txt	CreateFileA: Generic Read, Disposition: Open, ShareMode: Read
13:49:26.806	parent.exe (24216)	CreateFile, CreateFileMapping child.exe	CreateProcessA открывает образ child.exe и создаёт секцию (PAGE_EXECUTE)
13:49:26.813	parent.exe (24216)	Process Create child.exe	CreateProcessA: PID 22480, Command line: child.exe
13:49:26.813	child.exe (22480)	Process Start	Parent PID 24216, тот же текущий каталог
13:49:26.8298	child.exe (22480)	ReadFile input.txt	Offset 0, Length 82 – весь файл прочитан первым вызовом ReadFile

Время	Процесс (PID)	Событие	Комментарий
13:49:26.8305	child.exe (22480)	ReadFile input.txt – END OF FILE	Offset 82, Length 4096 (BUF_SIZE) – конец файла
13:49:26.834	child.exe (22480)	Process Exit	Exit Status 0
13:49:26.835	child.exe (22480)	CloseFile input.txt	закрыт последний дескриптор файла
13:49:26.839	parent.exe (24216)	Process Exit	Exit Status 0

Открытие файла. Файл input.txt открывает только родительский процесс; параметры события совпадают с аргументами CreateFileA в листинге 1.

Листинг 10 – Открытие входного файла родителем (фрагмент журнала Process Monitor)

```
13:49:26,8026065 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2\input.txt
    SUCCESS Desired Access: Generic Read, Disposition: Open,
    Options: Synchronous IO Non-Alert, Non-Directory File, Attributes: N,
    ShareMode: Read, AllocationSize: n/a, OpenResult: Opened
```

Создание дочернего процесса. Перед созданием процесса CreateProcessA открывает файл child.exe и отображает его в память (CreateFileMapping с PAGE_EXECUTE); результат «FILE LOCKED WITH ONLY READERS» – штатная информационная запись при создании секции образа. В событии Process Start потомка указан Parent PID 24216 – PID родителя.

Листинг 11 – Создание child.exe (фрагмент журнала Process Monitor)

```
13:49:26,8057005 parent.exe 24216 CreateFileMapping C:\Users\User\Desktop\lab2\
child.exe
    FILE LOCKED WITH ONLY READERS SyncType: SyncTypeCreateSection,
PageProtection: PAGE_EXECUTE
13:49:26,8131256 parent.exe 24216 Process Create C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS PID: 22480, Command line: child.exe
13:49:26,8131366 child.exe 22480 Process Start
    SUCCESS Parent PID: 24216, Command line: child.exe,
    Current directory: C:\Users\User\Desktop\lab2\
```

Чтение файла дочерним процессом. Главное наблюдение трассы: child.exe читает input.txt (ReadFile), но события CreateFile для этого файла у него нет – дочерний процесс файл не открывал, а получил дескриптор по наследству через hStdInput структуры STARTUPINFOA. Первый ReadFile вернул все 82 байта файла, второй (запрошено 4096 байт – BUF_SIZE в функции reader_getc) вернул END OF FILE, то есть 0 байт, после чего дочерний процесс выдал результат последней строки и завершился с кодом 0.

Листинг 12 – Чтение stdin и завершение child.exe (фрагмент журнала Process Monitor)

```
13:49:26,8297699 child.exe 22480 ReadFile C:\Users\User\Desktop\lab2\input.txt
    SUCCESS Offset: 0, Length: 82, Priority: Normal
13:49:26,8304610 child.exe 22480 ReadFile C:\Users\User\Desktop\lab2\input.txt
    END OF FILE Offset: 82, Length: 4 096
13:49:26,8339026 child.exe 22480 Process Exit
    SUCCESS Exit Status: 0, User Time: 0.0000000 seconds, Kernel Time: 0.0156250
seconds
13:49:26,8348838 child.exe 22480 CloseFile C:\Users\User\Desktop\lab2\input.txt
```

```
    SUCCESS
```

Событие CloseFile для input.txt появляется только после завершения дочернего процесса. Родитель закрыл свою копию дескриптора сразу после CreateProcessA (листинг 3), но файловый объект освобождается, когда закрыт последний ссылающийся на него дескриптор – дескриптор потомка, который система закрывает при завершении процесса. Родительский процесс завершился через 5 мс после дочернего (13:49:26.839) с кодом 0.

Операции чтения и записи канала pipe1 в журнале не отображены: анонимный канал не связан с файлом на диске, и его ввод-вывод Process Monitor в этом запуске не зарегистрировал. Не показано и чтение имени файла с консоли. Передача данных через канал подтверждается выводом программы (листинг 8): строки с PID дочернего процесса выведены родителем.

Выводы

В ходе работы на Win32 API реализовано взаимодействие двух процессов: parent.exe открывает файл, создаёт анонимный канал и запускает child.exe с перенаправленными стандартными потоками, child.exe построчно вычисляет сумму чисел типа float и передаёт результат родителю через канал. Корректность работы подтверждена тестами из таблицы 3, включая пустые строки, переполнение float, некорректные числа и отсутствие файла. Трасса Process Monitor подтверждает порядок системных вызовов: открытие файла родителем, создание процесса и чтение файла потомком через унаследованный дескриптор. Основная сложность – управление наследованием дескрипторов: если родитель не закроет свою копию записывающего конца канала, чтение из канала никогда не завершится; также важно читать канал до ожидания завершения потомка, чтобы избежать взаимоблокировки на заполненном буфере канала. Изучены функции CreatePipe, CreateProcess с STARTF_USESTDHANDLES, SetHandleInformation, ReadFile, WriteFile, WaitForSingleObject и анализ поведения процессов с помощью Process Monitor.

Приложение А. Исходный код программы

Листинг А.1 – Файл parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "common.h"

#define CHILD_EXE "child.exe"

static int read_file_name(char *buf, size_t size)
{
    printf("[parent PID %lu] Enter file name: ", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);

    if (fgets(buf, (int)size, stdin) == NULL) {
        fprintf(stderr, "[parent] ERROR: no file name (EOF)\n");
        return -1;
    }
    buf[strcspn(buf, "\r\n")] = '\0';
    if (buf[0] == '\0') {
        fprintf(stderr, "[parent] ERROR: empty file name\n");
        return -1;
```

```
    }
    return 0;
}

int main(void)
{
    char file_name[MAX_PATH];
    HANDLE hFile = INVALID_HANDLE_VALUE;
    HANDLE hPipeRead = NULL;
    HANDLE hPipeWrite = NULL;
    PROCESS_INFORMATION pi;
    STARTUPINFOA si;
    SECURITY_ATTRIBUTES sa;
    char cmd_line[] = CHILD_EXE;
    char buf[BUF_SIZE];
    HANDLE hStdout;
    DWORD child_code = 0;
    int result = EXIT_FAILURE;

    ZeroMemory(&pi, sizeof(pi));

    printf("[parent PID %lu] started\n", (unsigned long)GetCurrentProcessId());

    if (read_file_name(file_name, sizeof(file_name)) != 0) {
        return EXIT_FAILURE;
    }

    /* дескрипторы с этими атрибутами наследуются дочерним процессом */
    sa.nLength = sizeof(sa);
    sa.lpSecurityDescriptor = NULL;
    sa.bInheritHandle = TRUE;

    hFile = CreateFileA(file_name, GENERIC_READ, FILE_SHARE_READ, &sa,
                        OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hFile == INVALID_HANDLE_VALUE) {
        print_error("CreateFile");
        goto cleanup;
    }

    if (!CreatePipe(&hPipeRead, &hPipeWrite, &sa, 0)) {
        print_error("CreatePipe");
        goto cleanup;
    }
    /* конец для чтения остаётся только у родителя */
    if (!SetHandleInformation(hPipeRead, HANDLE_FLAG_INHERIT, 0)) {
        print_error("SetHandleInformation");
        goto cleanup;
    }

    /* stdin ребёнка - файл, stdout - pipe1 */
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
```

```
    si.dwFlags = STARTF_USESTDHANDLES;
    si.hStdInput = hFile;
    si.hStdOutput = hPipeWrite;
    si.hStdError = GetStdHandle(STD_ERROR_HANDLE);

    if (!CreateProcessA(NULL, cmd_line, NULL, NULL, TRUE, 0, NULL, NULL, &si,
&pi)) {
        print_error("CreateProcess(" CHILD_EXE ")");
        goto cleanup;
    }
    printf("[parent PID %lu] created child PID %lu\n",
           (unsigned long)GetCurrentProcessId(), (unsigned long)pi.dwProcessId);
    fflush(stdout);

    CloseHandle(pi.hThread);
    pi.hThread = NULL;

    /* закрываем свои копии, иначе ReadFile не дождётся конца канала */
    CloseHandle(hPipeWrite);
    hPipeWrite = NULL;
    CloseHandle(hFile);
    hFile = INVALID_HANDLE_VALUE;

    hStdout = GetStdHandle(STD_OUTPUT_HANDLE);
    if (hStdout == INVALID_HANDLE_VALUE) {
        print_error("GetStdHandle(STD_OUTPUT_HANDLE)");
        goto cleanup;
    }
    for (;;) {
        DWORD n = 0;
        if (!ReadFile(hPipeRead, buf, sizeof(buf), &n, NULL)) {
            if (GetLastError() == ERROR_BROKEN_PIPE) {
                break;  /* ребёнок закрыл канал */
            }
            print_error("ReadFile(pipe1)");
            goto cleanup;
        }
        if (n == 0) {
            break;
        }
        if (!write_all(hStdout, buf, n)) {
            print_error("WriteFile(stdout)");
            goto cleanup;
        }
    }

    if (WaitForSingleObject(pi.hProcess, INFINITE) == WAIT_FAILED) {
        print_error("WaitForSingleObject");
        goto cleanup;
    }
    if (!GetExitCodeProcess(pi.hProcess, &child_code)) {
        print_error("GetExitCodeProcess");
```

```
        goto cleanup;
    }
    printf("[parent PID %lu] child PID %lu exited with code %lu\n",
           (unsigned long)GetCurrentProcessId(), (unsigned long)pi.dwProcessId,
           (unsigned long)child_code);
    result = (child_code == 0) ? EXIT_SUCCESS : EXIT_FAILURE;

cleanup:
    if (hPipeWrite != NULL)            CloseHandle(hPipeWrite);
    if (hPipeRead != NULL)             CloseHandle(hPipeRead);
    if (hFile != INVALID_HANDLE_VALUE) CloseHandle(hFile);
    if (pi.hThread != NULL)            CloseHandle(pi.hThread);
    if (pi.hProcess != NULL) {
        WaitForSingleObject(pi.hProcess, INFINITE);
        CloseHandle(pi.hProcess);
    }
    return result;
}
```

Листинг А.2 – Файл child.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <stdarg.h>
#include "common.h"

#define TOKEN_MAX 64

typedef struct {
    HANDLE h;
    char buf[BUF_SIZE];
    DWORD pos;
    DWORD len;
} Reader;

typedef struct {
    float sum;
    int count;
    int error;
    char token[TOKEN_MAX + 1];
    int tlen;
    int too_long;
} LineState;

/* одним вызовом WriteFile, чтобы строка не смешалась с выводом родителя */
static void log_stderr(const char *fmt, ...)
{
    char msg[256];
    va_list ap;
    DWORD written;
```

```
    va_start(ap, fmt);
    int len = vsnprintf(msg, sizeof(msg), fmt, ap);
    va_end(ap);
    if (len < 0) {
        return;
    }
    if (len >= (int)sizeof(msg)) {
        len = (int)sizeof(msg) - 1;
    }
    if (!WriteFile(GetStdHandle(STD_ERROR_HANDLE), msg, (DWORD)len, &written,
NULL)) {
        /* stderr недоступен - сообщить об ошибке некуда */
    }
}

/* 1 - символ прочитан, 0 - конец файла, -1 - ошибка */
static int reader_getc(Reader *r, char *c)
{
    if (r->pos == r->len) {
        if (!ReadFile(r->h, r->buf, sizeof(r->buf), &r->len, NULL)) {
            if (GetLastError() == ERROR_BROKEN_PIPE) {
                return 0;
            }
            return -1;
        }
        r->pos = 0;
        if (r->len == 0) {
            return 0;
        }
    }
    *c = r->buf[r->pos++];
    return 1;
}

static void line_reset(LineState *st)
{
    st->sum = 0.0f;
    st->count = 0;
    st->error = 0;
    st->tlen = 0;
    st->too_long = 0;
}

static void finish_token(LineState *st, unsigned long line_no)
{
    char *end;
    float value;

    if (st->tlen == 0 && !st->too_long) {
        return;
    }
    st->token[st->tlen] = '\0';
```

```
    if (st->too_long) {
        log_stderr("[child PID %lu] line %lu: number is too long\n",
                   (unsigned long)GetCurrentProcessId(), line_no);
        st->error = 1;
    } else {
        value = strtof(st->token, &end);
        if (end == st->token || *end != '\0' || !isfinite(value)) {
            log_stderr("[child PID %lu] line %lu: invalid number '%s'\n",
                       (unsigned long)GetCurrentProcessId(), line_no, st->token);
            st->error = 1;
        } else {
            st->sum += value;
            st->count++;
        }
    }
    st->tlen = 0;
    st->too_long = 0;
}

static BOOL finish_line(LineState *st, HANDLE out, unsigned long line_no)
{
    char msg[256];
    int len;

    if (st->count == 0 && !st->error) {
        line_reset(st);
        return TRUE;  /* пустая строка */
    }

    if (st->error) {
        len = snprintf(msg, sizeof(msg), "[child PID %lu] line %lu: ERROR
(invalid input)\n",
                       (unsigned long)GetCurrentProcessId(), line_no);
    } else if (!isfinite(st->sum)) {
        len = snprintf(msg, sizeof(msg), "[child PID %lu] line %lu: ERROR (float
overflow)\n",
                       (unsigned long)GetCurrentProcessId(), line_no);
    } else {
        len = snprintf(msg, sizeof(msg), "[child PID %lu] line %lu: sum = %g\n",
                       (unsigned long)GetCurrentProcessId(), line_no, (double)st-
>sum);
    }
    line_reset(st);

    if (len < 0) {
        log_stderr("[child] ERROR: snprintf failed\n");
        return FALSE;
    }
    if (!write_all(out, msg, (DWORD)len)) {
        print_error("WriteFile(stdout)");
        return FALSE;
```

```
    }
    return TRUE;
}

int main(void)
{
    static Reader in;
    LineState st;
    HANDLE hOut;
    unsigned long line_no = 1;
    char c;
    int r;

    log_stderr("[child PID %lu] started\n", (unsigned
long)GetCurrentProcessId());

    /* stdin и stdout, которые задал родитель (файл и pipe1) */
    in.h = GetStdHandle(STD_INPUT_HANDLE);
    hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    if (in.h == INVALID_HANDLE_VALUE || in.h == NULL) {
        print_error("GetStdHandle(STD_INPUT_HANDLE)");
        return EXIT_FAILURE;
    }
    if (hOut == INVALID_HANDLE_VALUE || hOut == NULL) {
        print_error("GetStdHandle(STD_OUTPUT_HANDLE)");
        return EXIT_FAILURE;
    }
    in.pos = 0;
    in.len = 0;
    line_reset(&st);

    while ((r = reader_getc(&in, &c)) == 1) {
        if (c == ' ' || c == '\t' || c == '\r' || c == '\n') {
            finish_token(&st, line_no);
            if (c == '\n') {
                if (!finish_line(&st, hOut, line_no)) {
                    return EXIT_FAILURE;
                }
                line_no++;
            }
        } else if (st.tlen < TOKEN_MAX) {
            st.token[st.tlen++] = c;
        } else {
            st.too_long = 1;
        }
    }
    if (r < 0) {
        print_error("ReadFile(stdin)");
        return EXIT_FAILURE;
    }

    /* последняя строка без '\n' */
```

```
    finish_token(&st, line_no);
    if (!finish_line(&st, hOut, line_no)) {
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}
```

Листинг А.3 – Файл common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <windows.h>

#define BUF_SIZE 4096

void print_error(const char *context);
BOOL write_all(HANDLE h, const char *data, DWORD len);

#endif
```

Листинг А.4 – Файл common.c

```
#include <stdio.h>
#include "common.h"

/* Печатает текст ошибки по коду GetLastError() */
void print_error(const char *context)
{
    DWORD code = GetLastError();

    wchar_t wmsg[512];
    DWORD wlen = FormatMessageW(FORMAT_MESSAGE_FROM_SYSTEM |
FORMAT_MESSAGE_IGNORE_INSERTS,
                                NULL, code, 0, wmsg, 512, NULL);

    /* перекодируем в кодировку консоли */
    char msg[1024] = "unknown error";
    if (wlen > 0) {
        UINT cp = GetConsoleOutputCP();
        if (cp == 0) {
            cp = CP_ACP;
        }
        int n = WideCharToMultiByte(cp, 0, wmsg, (int)wlen, msg, (int)sizeof(msg)
- 1, NULL, NULL);
        if (n > 0) {
            msg[n] = '\0';
            while (n > 0 && (msg[n - 1] == '\r' || msg[n - 1] == '\n' || msg[n -
1] == ' ')) {
                msg[--n] = '\0';
            }
        }
    }
```

```
    fprintf(stderr, "[PID %lu] ERROR: %s: %s (code %lu)\n",
            (unsigned long)GetCurrentProcessId(), context, msg, (unsigned
long)code);
}

/* WriteFile может записать не всё сразу, поэтому пишем в цикле */
BOOL write_all(HANDLE h, const char *data, DWORD len)
{
    while (len > 0) {
        DWORD written = 0;
        if (!WriteFile(h, data, len, &written, NULL)) {
            return FALSE;
        }
        if (written == 0) {
            SetLastError(ERROR_WRITE_FAULT);
            return FALSE;
        }
        data += written;
        len -= written;
    }
    return TRUE;
}
```

Листинг А.5 – Файл Makefile

```
CC     = gcc
CFLAGS = -Wall -Wextra -O2

ifeq ($(OS),Windows_NT)
    RM = del /Q /F
else
    RM = rm -f
endif

all: parent.exe child.exe

parent.exe: parent.o common.o
    $(CC) $(CFLAGS) -o $@ $^

child.exe: child.o common.o
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c common.h
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    -$(RM) *.o *.exe

.PHONY: all clean
```

Приложение Б. Журнал Process Monitor

Приведены все события процессов parent.exe и child.exe, кроме 194 обращений к реестру (RegOpenKey, RegQueryValue, RegCloseKey, RegEnumValue) при загрузке системных библиотек. Блоки переменных окружения в событиях Process Start заменены на [...]. Порядок записей и значения остальных полей сохранены. Формат строки: время, процесс, PID, операция, путь, результат, подробности.

Листинг Б.1 – Журнал Process Monitor (parent.exe, child.exe)

```
13:49:15,5723064 parent.exe 24216 Process Start
    SUCCESS Parent PID: 26124, Command line: "C:\Users\User\Desktop\lab2\
parent.exe", Current directory: C:\Users\User\Desktop\lab2\, Environment: [...]
13:49:15,5723203 parent.exe 24216 Thread Create
    SUCCESS Thread ID: 25392
13:49:15,5740726 parent.exe 24216 Load Image C:\Users\User\Desktop\lab2\
parent.exe
    SUCCESS Image Base: 0x7ff78dd60000, Image Size: 0x16000
13:49:15,5742033 parent.exe 24216 Load Image C:\Windows\System32\ntdll.dll
    SUCCESS Image Base: 0x7ff921ac0000, Image Size: 0x268000
13:49:15,5758422 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2
    SUCCESS Desired Access: Execute/Traverse, Synchronize, Disposition: Open,
Options: Directory, Synchronous IO Non-Alert, Attributes: n/a, ShareMode: Read,
Write, AllocationSize: n/a, OpenResult: Opened
13:49:15,5763076 parent.exe 24216 Load Image C:\Windows\System32\kernel32.dll
    SUCCESS Image Base: 0x7ff91fae0000, Image Size: 0xcb000
13:49:15,5765932 parent.exe 24216 Load Image C:\Windows\System32\KernelBase.dll
    SUCCESS Image Base: 0x7ff91f030000, Image Size: 0x401000
13:49:15,5799520 parent.exe 24216 Load Image C:\Windows\System32\ucrtbase.dll
    SUCCESS Image Base: 0x7ff91f5b0000, Image Size: 0x14c000
13:49:15,5801120 parent.exe 24216 Thread Create
    SUCCESS Thread ID: 26044
13:49:15,5808763 parent.exe 24216 QueryNameInformationFile C:\Users\User\Desktop\
lab2\parent.exe
    SUCCESS Name: \Users\User\Desktop\lab2\parent.exe
13:49:26,8026065 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2\input.txt
    SUCCESS Desired Access: Generic Read, Disposition: Open, Options: Synchronous
IO Non-Alert, Non-Directory File, Attributes: N, ShareMode: Read, AllocationSize:
n/a, OpenResult: Opened
13:49:26,8033544 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open
Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete, AllocationSize:
n/a, OpenResult: Opened
13:49:26,8034302 parent.exe 24216 QueryBasicInformationFile C:\Users\User\
Desktop\lab2\child.exe
    SUCCESS CreationTime: 08.10.2026 14:06:31, LastAccessTime: 22.10.2026
13:46:58, LastWriteTime: 22.10.2026 13:45:35, ChangeTime: 22.10.2026 13:45:35,
FileAttributes: A
13:49:26,8034453 parent.exe 24216 CloseFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS
13:49:26,8039419 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open
Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete, AllocationSize:
n/a, OpenResult: Opened
```

```
13:49:26,8041376 parent.exe 24216 QueryBasicInformationFile C:\Users\User\
Desktop\lab2\child.exe
    SUCCESS CreationTime: 08.10.2026 14:06:31, LastAccessTime: 22.10.2026
13:46:58, LastWriteTime: 22.10.2026 13:45:35, ChangeTime: 22.10.2026 13:45:35,
FileAttributes: A
13:49:26,8041596 parent.exe 24216 CloseFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS
13:49:26,8046413 parent.exe 24216 Load Image C:\Windows\System32\sechost.dll
    SUCCESS Image Base: 0x7ff920270000, Image Size: 0xa8000
13:49:26,8056357 parent.exe 24216 CreateFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS Desired Access: Read Data/List Directory, Execute/Traverse, Read
Attributes, Synchronize, Disposition: Open, Options: Synchronous IO Non-Alert,
Non-Directory File, Attributes: N, ShareMode: Read, Delete, AllocationSize: n/a,
OpenResult: Opened
13:49:26,8057005 parent.exe 24216 CreateFileMapping C:\Users\User\Desktop\lab2\
child.exe
    FILE LOCKED WITH ONLY READERS SyncType: SyncTypeCreateSection,
PageProtection: PAGE_EXECUTE
13:49:26,8057356 parent.exe 24216 CreateFileMapping C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS SyncType: SyncTypeOther
13:49:26,8058410 parent.exe 24216 QuerySecurityFile C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS Information: Label
13:49:26,8059340 parent.exe 24216 QueryNameInformationFile C:\Users\User\Desktop\
lab2\child.exe
    SUCCESS Name: \Users\User\Desktop\lab2\child.exe
13:49:26,8063391 parent.exe 24216 QueryEAFile C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS
13:49:26,8131256 parent.exe 24216 Process Create C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS PID: 22480, Command line: child.exe
13:49:26,8131366 child.exe 22480 Process Start
    SUCCESS Parent PID: 24216, Command line: child.exe, Current directory: C:\
Users\User\Desktop\lab2\, Environment: [...]
13:49:26,8131539 child.exe 22480 Thread Create
    SUCCESS Thread ID: 6012
13:49:26,8139875 parent.exe 24216 QuerySecurityFile C:\Users\User\Desktop\lab2\
child.exe
    SUCCESS Information: Owner, Group, DACL, SACL, Label, Attribute, Process
Trust Label, 0x100
13:49:26,8148339 parent.exe 24216 QueryBasicInformationFile C:\Users\User\
Desktop\lab2\child.exe
    SUCCESS CreationTime: 08.10.2026 14:06:31, LastAccessTime: 22.10.2026
13:49:26, LastWriteTime: 22.10.2026 13:45:35, ChangeTime: 22.10.2026 13:45:35,
FileAttributes: A
13:49:26,8149061 parent.exe 24216 QueryBasicInformationFile C:\Users\User\
Desktop\lab2\child.exe
    SUCCESS CreationTime: 08.10.2026 14:06:31, LastAccessTime: 22.10.2026
13:49:26, LastWriteTime: 22.10.2026 13:45:35, ChangeTime: 22.10.2026 13:45:35,
FileAttributes: A
```

```
13:49:26,8149251 parent.exe 24216 QueryNameInformationFile C:\Users\User\Desktop\
lab2\child.exe
    SUCCESS Name: \Users\User\Desktop\lab2\child.exe
13:49:26,8181121 parent.exe 24216 CloseFile C:\Users\User\Desktop\lab2\child.exe
    SUCCESS
13:49:26,8182610 child.exe 22480 Load Image C:\Users\User\Desktop\lab2\child.exe
    SUCCESS Image Base: 0x7ff6f5930000, Image Size: 0x17000
13:49:26,8184252 child.exe 22480 Load Image C:\Windows\System32\ntdll.dll
    SUCCESS Image Base: 0x7ff921ac0000, Image Size: 0x268000
13:49:26,8210928 child.exe 22480 CreateFile C:\Users\User\Desktop\lab2
    SUCCESS Desired Access: Execute/Traverse, Synchronize, Disposition: Open,
Options: Directory, Synchronous IO Non-Alert, Attributes: n/a, ShareMode: Read,
Write, AllocationSize: n/a, OpenResult: Opened
13:49:26,8214899 child.exe 22480 Load Image C:\Windows\System32\kernel32.dll
    SUCCESS Image Base: 0x7ff91fae0000, Image Size: 0xcb000
13:49:26,8219752 child.exe 22480 Load Image C:\Windows\System32\KernelBase.dll
    SUCCESS Image Base: 0x7ff91f030000, Image Size: 0x401000
13:49:26,8270015 child.exe 22480 Load Image C:\Windows\System32\ucrtbase.dll
    SUCCESS Image Base: 0x7ff91f5b0000, Image Size: 0x14c000
13:49:26,8278375 child.exe 22480 Thread Create
    SUCCESS Thread ID: 32300
13:49:26,8293627 child.exe 22480 QueryNameInformationFile C:\Users\User\Desktop\
lab2\child.exe
    SUCCESS Name: \Users\User\Desktop\lab2\child.exe
13:49:26,8297699 child.exe 22480 ReadFile C:\Users\User\Desktop\lab2\input.txt
    SUCCESS Offset: 0, Length: 82, Priority: Normal
13:49:26,8304610 child.exe 22480 ReadFile C:\Users\User\Desktop\lab2\input.txt
    END OF FILE Offset: 82, Length: 4 096
13:49:26,8309448 child.exe 22480 CreateFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open
Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete, AllocationSize:
n/a, OpenResult: Opened
13:49:26,8309912 child.exe 22480 QueryBasicInformationFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS CreationTime: 02.10.2026 0:26:35, LastAccessTime: 22.10.2026
13:49:01, LastWriteTime: 02.10.2026 0:26:35, ChangeTime: 13.10.2026 22:51:20,
FileAttributes: A
13:49:26,8310273 child.exe 22480 CloseFile C:\Windows\System32\kernel.appcore.dll
    SUCCESS
13:49:26,8315566 child.exe 22480 CreateFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Desired Access: Read Data/List Directory, Execute/Traverse,
Synchronize, Disposition: Open, Options: Synchronous IO Non-Alert, Non-Directory
File, Attributes: n/a, ShareMode: Read, Delete, AllocationSize: n/a, OpenResult:
Opened
13:49:26,8316528 child.exe 22480 CreateFileMapping C:\Windows\System32\
kernel.appcore.dll
    FILE LOCKED WITH ONLY READERS SyncType: SyncTypeCreateSection,
PageProtection: PAGE_EXECUTE
13:49:26,8316864 child.exe 22480 CreateFileMapping C:\Windows\System32\
kernel.appcore.dll
```

```
    SUCCESS SyncType: SyncTypeOther
13:49:26,8319318 child.exe 22480 Load Image C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Image Base: 0x7ff91d950000, Image Size: 0x1b000
13:49:26,8322724 child.exe 22480 Load Image C:\Windows\System32\msvcrt.dll
    SUCCESS Image Base: 0x7ff9210b0000, Image Size: 0xa9000
13:49:26,8326358 child.exe 22480 CloseFile C:\Windows\System32\kernel.appcore.dll
    SUCCESS
13:49:26,8331999 child.exe 22480 Thread Exit
    SUCCESS Thread ID: 6012, User Time: 0.0000000, Kernel Time: 0.0156250
13:49:26,8333019 child.exe 22480 Thread Exit
    SUCCESS Thread ID: 32300, User Time: 0.0000000, Kernel Time: 0.0000000
13:49:26,8339026 child.exe 22480 Process Exit
    SUCCESS Exit Status: 0, User Time: 0.0000000 seconds, Kernel Time: 0.0156250
seconds, Private Bytes: 643 072, Peak Private Bytes: 643 072, Working Set:
4 571 136, Peak Working Set: 4 571 136
13:49:26,8343208 child.exe 22480 CloseFile C:\Users\User\Desktop\lab2
    SUCCESS
13:49:26,8348838 child.exe 22480 CloseFile C:\Users\User\Desktop\lab2\input.txt
    SUCCESS
13:49:26,8365653 parent.exe 24216 CreateFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Desired Access: Read Attributes, Disposition: Open, Options: Open
Reparse Point, Attributes: n/a, ShareMode: Read, Write, Delete, AllocationSize:
n/a, OpenResult: Opened
13:49:26,8365920 parent.exe 24216 QueryBasicInformationFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS CreationTime: 02.10.2026 0:26:35, LastAccessTime: 22.10.2026
13:49:26, LastWriteTime: 02.10.2026 0:26:35, ChangeTime: 13.10.2026 22:51:20,
FileAttributes: A
13:49:26,8366022 parent.exe 24216 CloseFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS
13:49:26,8367104 parent.exe 24216 CreateFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Desired Access: Read Data/List Directory, Execute/Traverse,
Synchronize, Disposition: Open, Options: Synchronous IO Non-Alert, Non-Directory
File, Attributes: n/a, ShareMode: Read, Delete, AllocationSize: n/a, OpenResult:
Opened
13:49:26,8367535 parent.exe 24216 CreateFileMapping C:\Windows\System32\
kernel.appcore.dll
    FILE LOCKED WITH ONLY READERS SyncType: SyncTypeCreateSection,
PageProtection: PAGE_EXECUTE
13:49:26,8367803 parent.exe 24216 CreateFileMapping C:\Windows\System32\
kernel.appcore.dll
    SUCCESS SyncType: SyncTypeOther
13:49:26,8369306 parent.exe 24216 Load Image C:\Windows\System32\
kernel.appcore.dll
    SUCCESS Image Base: 0x7ff91d950000, Image Size: 0x1b000
13:49:26,8371612 parent.exe 24216 Load Image C:\Windows\System32\msvcrt.dll
    SUCCESS Image Base: 0x7ff9210b0000, Image Size: 0xa9000
```

```
13:49:26,8374517 parent.exe 24216 CloseFile C:\Windows\System32\
kernel.appcore.dll
    SUCCESS
13:49:26,8379999 parent.exe 24216 Thread Exit
    SUCCESS Thread ID: 25392, User Time: 0.0000000, Kernel Time: 0.0312500
13:49:26,8380408 parent.exe 24216 Thread Exit
    SUCCESS Thread ID: 26044, User Time: 0.0000000, Kernel Time: 0.0000000
13:49:26,8392880 parent.exe 24216 Process Exit
    SUCCESS Exit Status: 0, User Time: 0.0000000 seconds, Kernel Time: 0.0312500
seconds, Private Bytes: 704 512, Peak Private Bytes: 704 512, Working Set:
5 001 216, Peak Working Set: 5 001 216
13:49:26,8397609 parent.exe 24216 CloseFile C:\Users\User\Desktop\lab2
    SUCCESS
```